

Case-Study Guide — Amii × ZeroKey ML Resident

Job ID: 1085 · **Companion to:** `amii-zerokey-ml-resident-technical.md` (§9 playbook, §10 Cases A–G)

Prepared: 2026-09-29 · **Format:** 10–20 minute interactive mini case study, no coding, tools optional

This guide has 13 practice scenarios (H–T) that don't repeat Cases A–G in the technical guide. Each one has a **model answer written as a system design**: likely answers to your clarifying questions, requirements, an architecture, the component choices and why, budgets, evaluation, rollout, and answers to the follow-up questions an interviewer is likely to ask. Each ends with a **60-second spoken summary** to rehearse.

How to use it: read the prompt, cover the answer, talk through your own design for 12–15 minutes with a timer, then compare. Don't memorize the answers. The interviewer is testing **how you reason**, and they will change the facts on you.

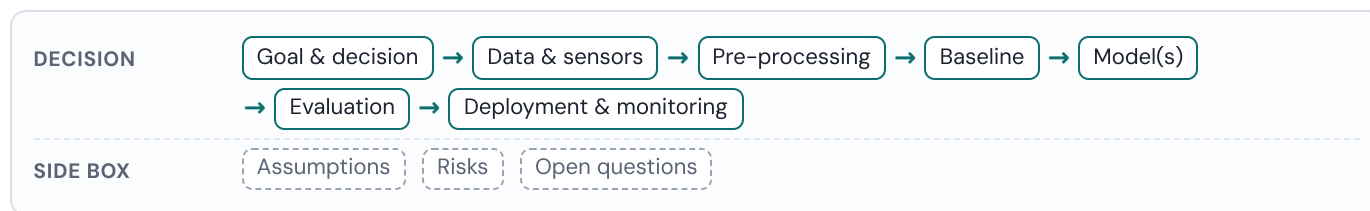
About the numbers. ZeroKey publishes ± 1.5 mm accuracy, 10,000+ events/s, and anomaly detection in under 500 ms. The **per-tag update rate isn't public**, so this guide assumes **20 Hz**. Every number marked (*assume*) is an assumption: say it out loud as one in the interview, and ask.

Part 1 — The playbook in one page

Time plan (15-minute case)

Minutes	Step	What you do
0–2	Clarify	Restate the goal. Ask 3–5 questions. Don't design yet.
2–4	Requirements & assumptions	Functional and non-functional requirements with numbers. State assumptions out loud.
4–6	Baseline	Rules, thresholds, or a classical method. "This is what any ML has to beat."
6–10	Architecture & model	Draw the pipeline. Choose the model and justify it. One alternative.
10–12	Evaluation	Leak-free splits, event-level metrics, success criterion.
12–14	Deployment, risks, first week	Latency, edge vs. server, monitoring, drift, privacy, what you'd do in week 1.
rest	Their questions	Welcome pushback. Adapt out loud.

The skeleton to draw every time



The system-design answer template

Use these headings for every scenario, in this order:

1. **Requirements.** Functional (what it outputs, to whom) and non-functional (latency, false-alarm budget, scale, privacy, hardware).

2. **Architecture.** The pipeline, and what runs at the **edge** vs. on the **server**.
3. **Components.** For each box: the method, why, and the simplest version that works.
4. **Budgets.** Latency, compute, alarm rate, labelling effort. Put numbers on them.
5. **Evaluation.** Offline (splits, metrics) and online (shadow mode, A/B, operator feedback).
6. **Rollout.** Week 1, month 1, months 2–6.
7. **Risks.** What breaks first, and what you'd do about it.

Clarifying questions that work for any case

1. What decision does the output drive, and who acts on it?
2. How fast must it respond (real-time alert vs. end-of-shift report)? On what hardware?
3. What data do we have, at what rate? What labels exist, and how many positives?
4. What does a false alarm cost compared with a miss? Is there an alarm budget?
5. One cell, or many sites? Privacy and explainability constraints?

If they won't answer: "Then I'll assume X, and I'll point out where the design changes if that's wrong."

Part 2 — Scenarios

Index

#	Scenario	Data	Main skills tested	Difficulty
H	Is the badge actually being worn?	RTLS	Anomaly detection, physics priors, base rates	★★
I	Discover work cycles with no SOP labels	RTLS	Unsupervised segmentation, motif discovery	★★★
J	Sensor fault or real anomaly?	RTLS + system logs	Data quality, Kalman innovations, root cause	★★
K	Plant A model, Plant B deployment	RTLS	Distribution shift, domain adaptation, few labels	★★★
L	Right part from the right bin?	RTLS + pick list	Spatial association, uncertainty, thresholds	★★
M	Man-down / fall detection for lone workers	RTLS	Rare events, alarm budgets, latency	★★
N	Novice vs. expert: skill assessment and coaching	RTLS	Trajectory similarity, metric learning, fairness	★★★
O	Does motion predict downstream defects?	RTLS + QC outcomes	Weak labels, causality, confounding	★★★
P	Tool-angle check from two tags	RTLS	3D geometry, rotations, noise propagation	★★
Q	PPE compliance from cameras	Video	Detection, tracking, class imbalance, privacy	★★
R	New product, 20 defect images	Images	Anomaly detection, few-shot, lighting	★★

#	Scenario	Data	Main skills tested	Difficulty
S	AGV fleet congestion forecasting	RTLS (vehicles + people)	Multi-agent forecasting, graphs	★★★
T	"Our model is 98% accurate" — find the problem	Any	Leakage, evaluation, critical thinking	★★

Rehearse first: T (pure critical thinking, a very likely style), I (hardest; shows depth with unlabelled data), Q or R (covers the computer-vision main topic), then K.

H. Is the badge actually being worn?

Prompt: "Safety analytics depend on workers wearing their badge and wrist tags. Some workers leave the badge on the bench or hang it on a cart. How would you detect a tag that isn't being worn?"

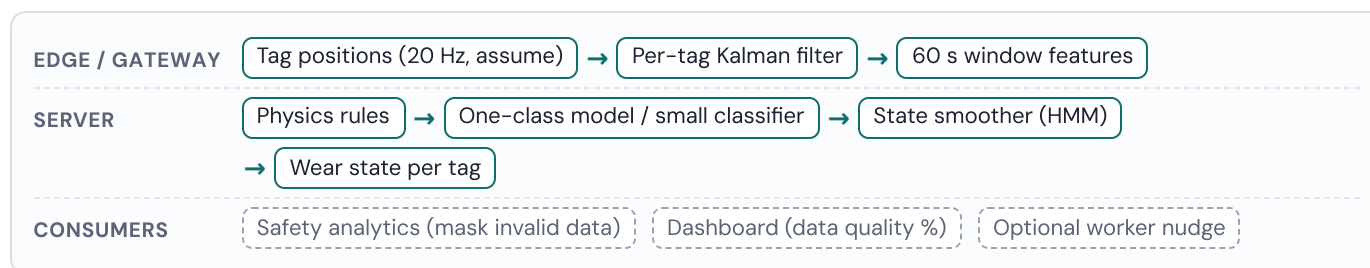
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Which tags does each worker carry?	Badge on a lanyard, plus a wristband on the dominant hand.	Two tags let you check they move together. Badge-only means relying on micro-motion and height alone.
Real-time alert, or a data-quality flag?	Mostly a data-quality flag, with a nudge to the worker after a few minutes.	Latency can be minutes, so you can use long windows (60 s) and be conservative.
Is this used for discipline?	Not intended; the customer wants valid safety data.	Design the output as "data invalid" on dashboards, not "worker X broke a rule."
Any labelled examples?	No.	Collect staged data; rely on physics rules first.

Requirements

- **Functional:** for each tag, a per-minute state: worn, not worn (stationary), not worn (on moving object), uncertain. Downstream safety analytics ignore data from tags that aren't worn.
- **Non-functional:** decision within 2–5 minutes; fewer than 1 false "not worn" flag per worker per week; runs on the server (no edge need); no individual-level reporting by default.

Architecture



Components

- **Features per 60 s window:**
 - *Micro-motion:* position standard deviation and spectral energy at 0.1–1 Hz (breathing and postural sway). A tag on a bench shows only sensor noise (≈ 1.5 mm); a worn one shows mm-to-cm sway even when the person stands still.
 - *Height* in the cell frame: chest height (~ 1.2 – 1.5 m) vs. bench (~ 0.9 m) or cart height.
 - *Gait:* periodicity at ~ 1.5 – 2 Hz step frequency while moving.
 - *Coupling:* badge–wristband distance ($\leq \sim 0.8$ m when worn) and correlation of their velocities.
- **Rules first:** "variance $\approx$ sensor noise for > 2 min" $\rightarrow$ stationary, not worn. "Moving smoothly without gait periodicity and no wrist coupling" $\rightarrow$ on a cart. These probably catch 80% of cases.

- **Learned layer:** a one-class model (e.g. isolation forest or Gaussian model) fitted on "worn" windows, or a small GBDT trained on staged data covering bench, cart, pocket, sitting, and standing still.
- **Smoother:** a 2-state HMM (worn / not worn) over consecutive windows, so a single odd window doesn't flip the state.

Budgets

- Compute: trivial (features per tag per minute). Hundreds of tags on one server core.
- Labelling: one staged session of about 2 hours with 5–10 people covering each "not worn" mode and hard normals (sitting, standing still at a bench).

Evaluation

- Staged recordings with ground truth, **split by person**.
- Metrics: per-minute precision and recall for "not worn"; false flags per worker per week on a week of normal data (every flag there is false if spot-checks confirm the badges were worn).
- Online: fraction of data flagged per site; spot-check a sample of flags with supervisors.

Rollout

- **Week 1:** stationary-tag recordings to measure real noise; staged session.
- **Month 1:** rules only, running silently; compare flags with spot checks.
- **Month 2+:** add the learned layer where rules are ambiguous (carts, sitting).

Follow-up questions, with answers

- **"A worker sits still at a desk for 10 minutes."** → Height drops and motion is small, but breathing sway remains above sensor noise, and the wrist moves. Use micro-motion and coupling, not height alone.
- **"Why not just ask workers to tap in?"** → Good idea; a procedural fix is cheaper. The detector still measures whether the procedure works. Challenging the framing is part of a good answer.
- **"Isn't this surveillance?"** → Default to aggregate data-quality reporting, agree the use with worker representatives, and keep the purpose to "valid safety data."
- **"What about a badge in a pocket?"** → Still worn, still moves with the body; height is lower. Treat as worn but lower confidence for posture-dependent analytics.

Spoken summary (60 seconds)

"A worn badge is never perfectly still: breathing and sway show up as millimetre motion well above the 1.5 mm sensor noise, and it moves with the wristband. So I'd start with physics rules on 60-second windows: micro-motion, height, gait periodicity, and badge-wrist coupling. That catches the bench case. The cart case is harder, so I'd add a small classifier trained on a staged session, with an HMM smoother. It runs on the server with minutes of latency, and the output is a data-quality mask for the safety analytics, not a disciplinary report."

Your link: McGill IMU gesture work (separating intended motion from idle); micro-motion is a spectral problem, like your DSP work.

I. Discover work cycles with no SOP labels

Prompt: "A customer installs RTLS on a line and wants cycle-time and step breakdowns, but they have no documented SOP and no labels. What would you do?"

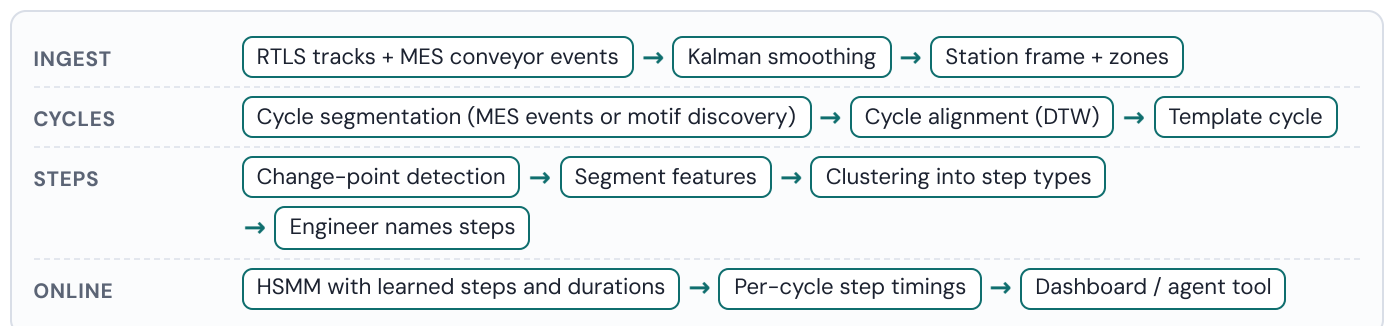
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
One operator per station?	Mostly yes; some shared stations.	Start with single-operator stations; handle shared ones later with tag identity.
How repetitive is the work?	Cycles of 1–3 minutes, fairly repetitive.	Periodicity methods will work; cycle-level alignment is feasible.
What output do they want?	Average cycle time, step breakdown, and where time is lost.	Need step segmentation, not just cycle counting.
Any event signals?	Conveyor index events from the MES.	Gives cycle boundaries for free; use them.
Can an engineer give an hour to label?	Yes, a few hours.	Use for naming discovered steps and validating.

Requirements

- **Functional:** per station, a list of discovered steps with names (after engineer review), per-cycle step durations, cycle-time distribution, and top sources of variation.
- **Non-functional:** batch processing (daily), no real-time need; explainable to process engineers; works on a new line within 1–2 weeks of data.

Architecture



Components

- **Pre-processing:** Kalman smoothing, transform to the station frame, define zones (bins, fixture, tool rack, conveyor) from dwell clusters.
- **Cycle segmentation:** cut at MES conveyor events if available. Otherwise use **autocorrelation** of the wrist trajectory for the dominant period and **matrix-profile motif discovery** to find repeating sub-sequences.
- **Cycle alignment:** DTW between cycles and **DTW barycenter averaging** for a template cycle, so each cycle can be mapped onto the template's timeline.
- **Step discovery:** change points (PELT) on speed and zone within the template; cluster segments by location, duration and motion shape; show clusters to the engineer as heatmaps and short replays to name

them.

- **Online segmentation:** an **HSMM** (hidden semi-Markov model) whose states are the named steps with duration distributions. Robust to small variations; gives step timings for every new cycle.
- **Later:** self-supervised embeddings (TS2Vec) instead of hand features if the hand features plateau.

Budgets

- Data: 1–2 weeks of normal production per station.
- Engineer time: ~1 hour per station to name and check steps.
- Compute: offline batch; DTW is $O(n \cdot m)$ per pair, so use a Sakoe–Chiba band and a sample of cycles for the template.

Evaluation

- **Cycle time:** compare with a stopwatch study or MES timings (target error < 5%).
- **Step boundaries:** an engineer labels ~1 hour; measure boundary error in seconds and segmental F1.
- **Stability:** the same steps should be discovered on different days and shifts.

Rollout

- **Week 1:** collect data; cycle segmentation; show cycle-time distribution (quick value).
- **Weeks 2–3:** step discovery with engineer review.
- **Month 2:** HSMM online; deviation detection (steps skipped, extra steps, long steps).

Follow-up questions, with answers

- **"The operator batches work — does two parts, then two assemblies."** → Cycles aren't strictly periodic; use event-based segmentation and a model that allows optional and repeated steps (HSMM with flexible transitions, or a grammar).
- **"How do you choose the number of step clusters?"** → BIC or silhouette as a guide, then engineer review. Start coarse; split steps only where engineers care.
- **"Why not deep learning?"** → No labels, and engineers need explanations. Deep models come in later as feature extractors.
- **"How do you know discovered steps are meaningful?"** → They should correspond to locations and tools the engineer recognizes, and be stable across days.

Spoken summary (60 seconds)

"With no labels, I'd use the structure of repetitive work. First cut the stream into cycles, from MES events if available, otherwise autocorrelation and motif discovery. Align cycles with DTW to build a template, find step boundaries with change-point detection, and cluster segments into step types that an engineer names in about an hour. Then an HSMM gives step timings for every new cycle. I'd validate against a stopwatch study and a small labelled sample, and the quick win in week one is the cycle-time distribution."

Your link: MAS sewing-floor IoT tracking repetitive operator motion; DTW experience from JAES; the McGill hierarchical FSM.

J. Sensor fault or real anomaly?

Prompt: "OmniVisor flags anomalies, but engineers complain many are caused by the positioning system itself, not the process. How would you separate sensor problems from real process anomalies?"

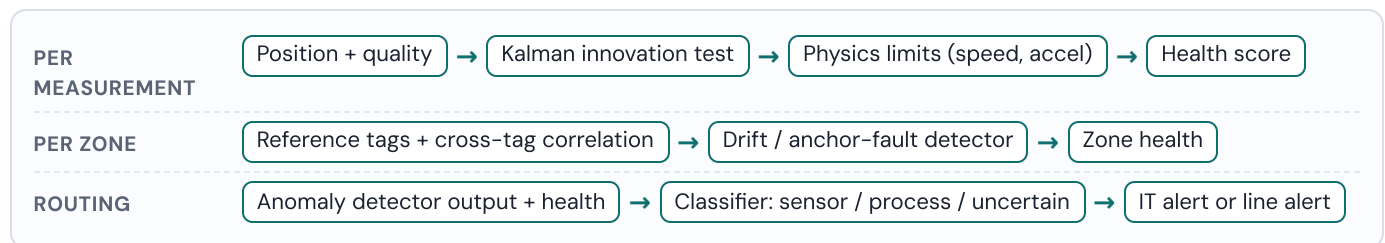
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
What do false anomalies look like?	Position jumps, frozen positions, slow drift in one area.	Build a fault taxonomy and a detector per type.
Do we have anchor-level data?	Possibly; at least a quality value per position.	With anchor residuals you can find the bad anchor; without, use cross-tag correlation.
Temperature data?	HVAC data exists but isn't joined.	Join it: $1^{\circ}\text{C} \approx 18 \text{ mm}$ at 10 m, so drift can follow temperature.
Fixed reference tags?	Not currently.	Recommend installing one per zone: cheap and very informative.

Requirements

- **Functional:** every position gets a health score; every anomaly is labelled `sensor`, `process`, or `uncertain`; sensor issues go to IT/maintenance, process anomalies to the line.
- **Non-functional:** health scoring must add $< 50 \text{ ms}$ to the $< 500 \text{ ms}$ anomaly budget; scales to 10,000+ events/s; explainable ("anchor 7 degraded since 14:02").

Architecture



Components

- **Fault taxonomy:**
 - Multipath or occlusion outliers: single-sample jumps faster than a person can move.
 - Dropouts: gaps.
 - Drift: slow offset shared by tags in one area (temperature, a moved anchor).
 - Anchor fault: all tags relying on one anchor degrade together.
 - Frozen tag: identical coordinates repeated (battery, firmware).
- **Per-measurement checks:** the Kalman innovation test (reject if $d^2 = y^T S^{-1} y > \chi^2_3(0.99) \approx 11.34$); speed and acceleration limits per tag type. Track the **normalized innovation squared** over time: if it's consistently too high, either the sensor or the filter's model is wrong.
- **Per-zone checks:** a **fixed reference tag** in each zone is the gold standard (any motion it shows is sensor error). Without one, look for tags in one area shifting together.

- **Anchor attribution:** if per-anchor range residuals from multilateration are available, the anchor with persistently large residuals is the culprit.
- **Routing:** process-anomaly alerts are suppressed or downgraded when the local health is poor.

Budgets

- Latency: innovation test and limits are $O(1)$ per measurement; well within 50 ms.
- Hardware: one reference tag per zone (a small cost compared with the time engineers lose on false anomalies).

Evaluation

- **Fault injection:** add synthetic jumps, drift, dropouts, and frozen segments to clean recordings; measure detection rate and time to detect per fault type.
- **Historical review:** engineers label a week of past anomalies as sensor or process; measure how many sensor-caused alerts the router would have suppressed and how many real ones it would have wrongly suppressed.
- **Online:** engineers' "false alarm" feedback rate before and after.

Rollout

- **Week 1:** install reference tags in two zones; review a week of past anomalies with engineers.
- **Month 1:** health scores in shadow mode; tune thresholds.
- **Month 2:** turn on routing; monitor that no real process anomalies were suppressed.

Follow-up questions, with answers

- **"No anchor-level data."** → Rely on reference tags and cross-tag correlation; ask ZeroKey's firmware team whether residuals can be exposed.
- **"Could a real event look like a sensor fault?"** → Yes: a dropped tool really does fall fast. Keep physics limits per tag type, and never delete data, only downgrade confidence.
- **"Why does temperature matter?"** → Ultrasound speed changes about 0.6 m/s per °C, so a 1 °C error is $\approx$ 0.18% range error, $\approx$ 18 mm at 10 m. That's large compared with 1.5 mm accuracy.

Spoken summary (60 seconds)

"I'd treat the positioning system as something to monitor, not assume. Every measurement gets a health score from a Kalman innovation test and physics limits; every zone gets a health score from a fixed reference tag and cross-tag correlation, which catches drift and anchor faults. Anomalies are then routed: sensor issues to maintenance, process anomalies to the line, and process alerts are downgraded where health is poor. I'd validate by injecting synthetic faults and by having engineers label a week of past anomalies."

Your link: MAS machine monitoring; validating systems before trusting their outputs.

K. Plant A model, Plant B deployment

Prompt: "Your risk model works well at the pilot plant. A new customer wants it deployed at a plant with a different layout, different products, and different workers. You get almost no labels at the new site. How do you approach it?"

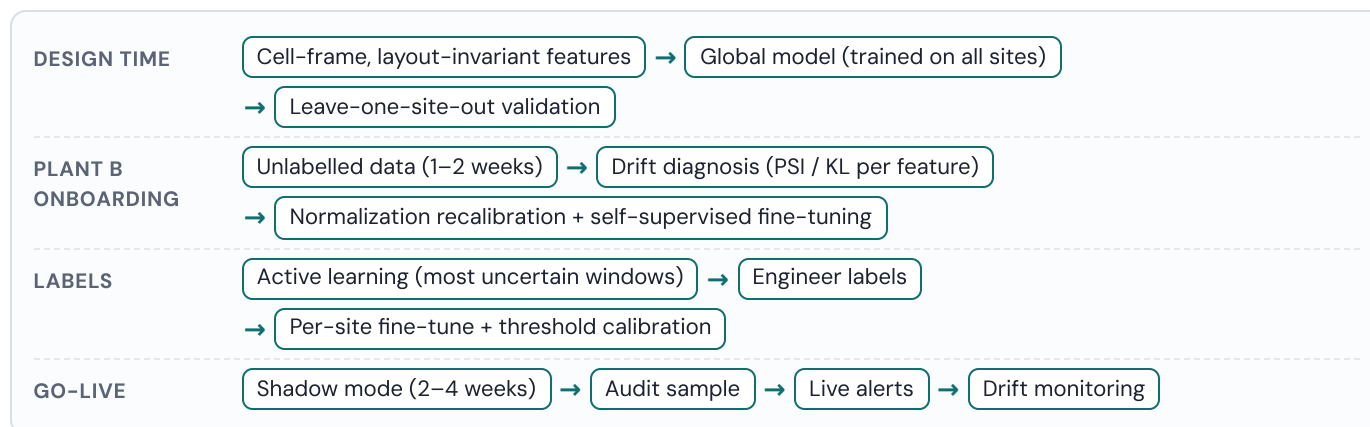
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Same risk definitions?	Yes, the same 3 risk types.	The label space is shared; the problem is covariate and label shift.
Same tag placement?	Yes, badge plus wristband.	Features are comparable; no sensor-level adaptation needed.
Time before go-live?	6–8 weeks.	Room for a 2–4 week shadow period.
Can engineers label anything?	A few hours per cell.	Active learning to spend those hours well.

Requirements

- **Functional:** the same risk alerts at Plant B, with per-site thresholds.
- **Non-functional:** precision and recall at Plant B within ~10% of Plant A before go-live; false alarms within the site's alarm budget; < 4 hours of labelling per cell.

Architecture



Components

- **Invariant features by design:** coordinates in the **work-cell frame**, not facility coordinates; relative quantities (wrist–badge distance, worker–robot distance, speeds, heights above floor); zone *types* rather than zone IDs.
- **Diagnose the shift:** compare feature distributions between plants (PSI or KL per feature). Separate *covariate shift* (layout, products), *label shift* (different base rates), and *concept shift* (different SOP means different "correct" behaviour).
- **Unsupervised adaptation:** recalibrate normalization statistics on Plant B; continue self-supervised pre-training on Plant B's unlabelled data; domain-adversarial training if the shift is large.

- **Few labels, spent well:** active learning picks the most uncertain or most novel windows for engineers.
- **Threshold calibration:** base rates differ, so set the threshold per site to meet its alarm budget.

Budgets

- Data: 1–2 weeks of unlabelled Plant B data before fine-tuning.
- Labels: ≤ 4 hours per cell.
- Shadow period: 2–4 weeks.

Evaluation

- **Before Plant B exists:** leave-one-site-out (or leave-one-line-out) validation. This is the honest estimate of transfer performance, and it tells you which features don't transfer.
- **At Plant B:** a small labelled audit set; event-level precision and recall; false alarms per shift during shadow mode.

Rollout

- **Weeks 1–2:** collect data, diagnose shift.
- **Weeks 3–4:** adapt, active learning, calibrate.
- **Weeks 5–8:** shadow mode, audit, go-live decision with the customer.

Follow-up questions, with answers

- **"One global model or one per site?"** → A global model plus per-site calibration and light fine-tuning. Per-site models from scratch don't scale and waste data.
- **"When do you retrain?"** → When drift monitoring or the alert rate moves beyond agreed limits, or operator feedback on false alarms rises.
- **"What if Plant B's SOP defines a different correct order?"** → That's concept shift: the sequence-compliance part must be reconfigured from Plant B's SOP, not learned from Plant A.

Spoken summary (60 seconds)

"I'd design for transfer from the start: features in the work-cell frame and relative quantities, not facility coordinates, and I'd measure transfer honestly with leave-one-site-out validation. At Plant B I'd collect a couple of weeks of unlabelled data, diagnose which features shifted, recalibrate and fine-tune self-supervised, then spend a few hours of engineer labelling through active learning. Thresholds are set per site for its alarm budget, and nothing goes live until a two-to-four-week shadow period meets the agreed metrics."

Your link: MAS rollouts across factories on three continents.

L. Right part from the right bin?

Prompt: "Operators pick parts from a rack of 24 small bins, 10 cm apart. Picking the wrong part causes defects. Using wrist tags, how would you verify every pick?"

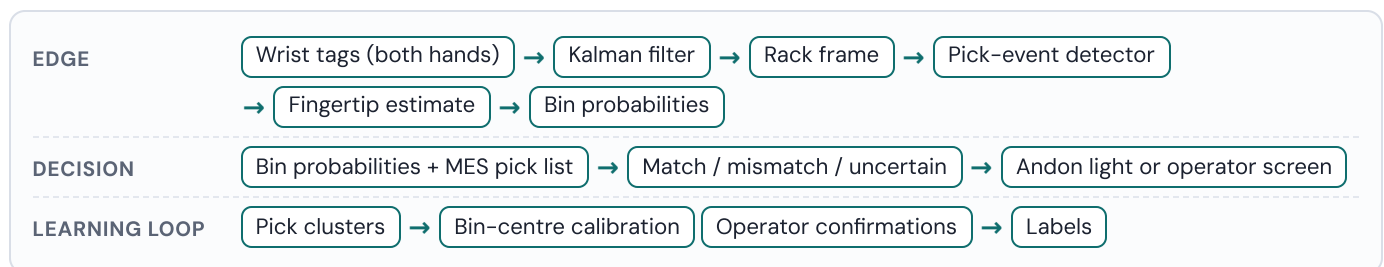
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Bin layout?	4 rows × 6 columns, 10 cm centre spacing, 15 cm deep.	Neighbour confusion is the main risk; 3D geometry needed.
Expected pick list per unit?	Yes, from the MES, per unit.	Verification = compare detected bin to expected bin.
When must it alert?	Before the part is installed (a few seconds).	Budget ≈ 1 s from pick to alert.
Which hand?	Either hand; wristbands on both.	Associate picks per hand.

Requirements

- **Functional:** for each pick: detected bin with probability, match / mismatch / uncertain vs. the pick list; alert on mismatch.
- **Non-functional:** alert within 1 s of the hand leaving the bin; wrong-bin recall $\geq 99\%$ (*target to agree*); false stops below an agreed budget (e.g. < 1 per shift); runs at the edge gateway.

Architecture



Components

- **Why it's feasible but not trivial:** bins are 10 cm apart and the sensor is ± 1.5 mm, so the *sensor* isn't the problem. The problem is that a wrist tag is ~15–20 cm from the fingertips, and wrist orientation varies.
- **Pick-event detection:** hand enters a bin's volume, slows, dwells ~0.3–1 s, retracts. Rules first (zone + speed minimum + dwell); a small classifier afterwards to reject "reaching past a bin."
- **Fingertip estimate:** extrapolate from the wrist along the approach direction (the velocity vector just before the slowdown) by a learned offset. Two tags on a glove would remove most of the ambiguity if the customer accepts it.
- **Bin association:** a probability per bin (e.g. softmax over negative distances with a learned scale), not a hard assignment.
- **Decision:** mismatch if the expected bin's probability is below a threshold; "uncertain" if two neighbouring bins are close; uncertain picks ask the operator to confirm on the screen (the confirmations become labels).

- **Geometry calibration:** the rack isn't exactly where the CAD says; estimate bin centres from clusters of pick positions over a day.

Budgets

Stage	Latency (target)
Positioning + Kalman	≤ 100 ms (<i>assume</i>)
Pick-event end detection	≤ 300 ms after retraction
Association + decision	< 10 ms
Alert display	≤ 200 ms
Total	< 1 s

Evaluation

- Staged picks with ground truth, including deliberate wrong picks and neighbour bins; split by operator.
- Confusion matrix between neighbouring bins; wrong-pick recall; false-stop rate per shift; latency distribution (p95).

Rollout

- **Week 1:** record picks; calibrate bin centres; measure neighbour confusion.
- **Month 1:** shadow mode against the pick list; operators unaware.
- **Month 2:** live alerts with the "uncertain → confirm" flow.

Follow-up questions, with answers

- **"Pick-to-light already exists."** → Pick-to-light tells the operator what to pick; this verifies what they actually picked. They complement each other.
- **"Operator grabs from the wrong bin, notices, and puts it back."** → Track the full sequence: pick → return to the same bin → new pick. Only alert on what reaches the assembly.
- **"Bins are 5 cm apart instead."** → Wrist-only becomes unreliable; recommend a fingertip or glove tag, or accept "uncertain" and confirm.

Spoken summary (60 seconds)

"The sensor is precise enough; the real error is the offset between the wrist tag and the fingertips. So I'd detect pick events from dwell and speed in each bin's volume, estimate the fingertip along the approach direction, and output a probability per bin. That's compared with the MES pick list at the edge, with an alert within a second. Ambiguous picks ask the operator to confirm, which also creates labels, and bin positions are calibrated from the pick data itself. The metric that matters most is the false-stop rate."

Your link: MAS care-label QC: verification with an explainable overlay and a PLC reject.

M. Man-down / fall detection for lone workers

Prompt: "A customer has workers in remote parts of a warehouse. They want an alert if someone falls or collapses. Badge tags only. Design it."

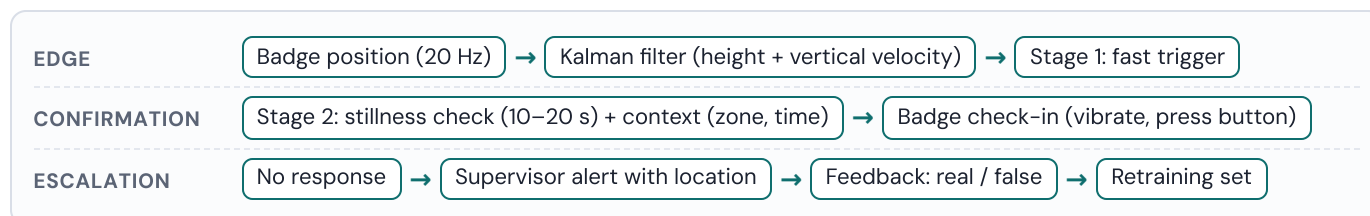
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Required response time?	Alert within 30 s.	Room for a confirmation window.
Who receives it?	Floor supervisor's phone.	Every false alarm interrupts a person; alarm budget matters.
Any real fall data?	None.	Staged falls plus lots of normal data.
Can the badge vibrate or have a button?	Yes, it has a button.	Add a "are you OK?" check-in before escalating.

Requirements

- **Functional:** detect falls and collapses; escalate to a supervisor with the location.
- **Non-functional:** alert ≤ 30 s after the event; recall on staged falls $\geq 95\%$ (*target to agree*); false alarms < 1 per 100 worker-shifts (*assume*); runs at the edge so it works if the server link is down.

Architecture



Components

- **The base-rate problem, stated early:** falls are extremely rare, and bending, kneeling and sitting happen hundreds of times per shift. Even a 1% false-trigger rate on bend-downs means many false alarms per day. That's why the design has stages.
- **Stage 1 — trigger:** badge height drops from ~ 1.3 m to < 0.5 m within ~ 1 s, with a vertical-speed peak above normal kneeling speeds.
- **Stage 2 — confirmation:** little gross motion for 10–20 s at the low height. A person kneeling to pick moves; a collapsed person doesn't (but still breathes, so the badge isn't perfectly still).
- **Context:** in zones with low shelves, raise the thresholds; in open aisles, lower them.
- **Check-in:** vibrate the badge; if the worker presses the button, cancel. No response $\rightarrow$ alert.
- **Learned layer:** a small classifier (GBDT on trigger-window features) trained on staged falls and hard negatives, only if the rules don't meet the alarm budget. Public fall datasets (IMU-based) can help pre-train, with care because the sensors differ.

Budgets

Stage	Time
Trigger	≤ 1 s after impact
Stillness confirmation	10–20 s
Check-in window	≤ 10 s
Total to alert	≤ 30 s

Evaluation

- **Staged falls** (with safety mats and a protocol) from several people and directions; recall must be near 100%.
- **Hard negatives:** kneeling, sitting, lying under a machine for maintenance, dropping the badge.
- **False alarms** measured on weeks of normal operation, per worker–shift.
- Time-to-alert distribution.

Rollout

- **Week 1:** staged fall and hard-negative session; measure normal bend-down rates per zone.
- **Month 1:** shadow mode; review every trigger.
- **Month 2:** live with check-in; weekly review of false alarms.

Follow-up questions, with answers

- **"The badge fell, not the person."** → A dropped badge shows no breathing micro-motion afterwards; a person on the floor does. The check-in resolves it either way; treat as "check on the worker."
- **"Recall must be 100%."** → Not achievable; say so, and let the customer choose the operating point on the recall vs. false-alarm curve, with the check-in reducing the cost of false alarms.
- **"Why not a wrist tag or IMU?"** → They'd help a lot (impact acceleration), and I'd recommend them; the design above works with the badge only, as asked.

Spoken summary (60 seconds)

"The core issue is the base rate: falls are rare and bending down is constant, so a one-stage detector would drown supervisors in false alarms. I'd use two stages at the edge: a fast trigger on a sudden drop in badge height, then a 10-to-20-second stillness confirmation, then a check-in on the badge, and only then a supervisor alert, all within 30 seconds. Training data comes from staged falls plus lots of hard negatives like kneeling and sitting, and the customer chooses the operating point."

Your link: onset detection and real-time latency budgets in JAES.

N. Novice vs. expert: skill assessment and coaching

Prompt: "A customer wants to shorten onboarding. Can we use RTLS data from experienced operators to assess new operators and coach them?"

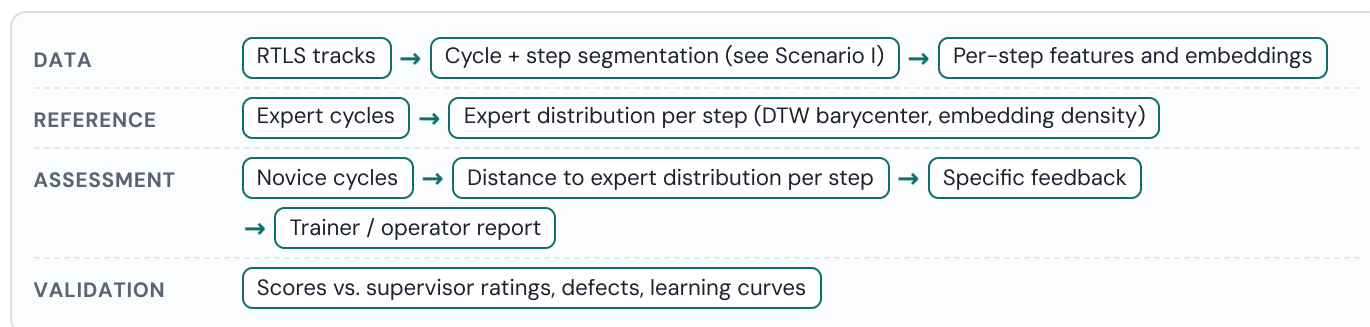
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
What counts as skill?	Speed and quality; fewer mistakes.	Multi-dimensional score, not just speed.
Outcome labels?	Supervisor ratings; defect records per operator.	Validate the score against these.
Who sees the scores?	Trainers and the operators themselves.	Coaching tool, not a ranking for management.
How many experts?	10–20 per line.	Enough to model a distribution of expert motion, not one "golden" operator.

Requirements

- **Functional:** per novice, per step: how their motion differs from the expert distribution, with specific feedback ("step 3 takes 40% longer; you walk to the tool rack twice").
- **Non-functional:** end-of-shift report; explainable; fair across body sizes and handedness; opt-in for workers.

Architecture



Components

- **Define skill with the customer:** cycle time, variation between cycles, path efficiency (path length ÷ straight-line distance), unnecessary motions, sequence errors, ergonomic proxies.
- **Expert reference:** a *distribution*, because experts differ. Per step: DTW barycenter plus spread, or a density model in an embedding space.
- **Metric learning (later):** a siamese or contrastive network trained so that embeddings separate experience levels. Only keep it if the distance predicts outcomes (defects, ratings), not just speed.
- **Feedback generator:** turn the largest differences into plain-language, step-level feedback; show the novice's path overlaid on the expert's.

Evaluation

- Does the score improve over a novice's first weeks (a learning curve)?
- Correlation with supervisor ratings and defect rates.
- Test–retest stability (the same operator on different days gets similar scores).
- Fairness: compare score distributions across handedness and body size among *equally rated* operators.

Rollout

- **Month 1:** step segmentation and expert reference on one line.
- **Month 2:** pilot with 5–10 novices and their trainers; collect feedback on usefulness.
- **Month 3+:** measure onboarding time vs. a previous cohort.

Follow-up questions, with answers

- **"Isn't this just speed?"** → No: faster isn't better if quality or safety drops. That's why it's multi-dimensional and validated against defects.
- **"A left-handed expert looks like an outlier."** → Mirror trajectories where the station allows, or model left- and right-handed references separately; always validate against outcomes.
- **"Will workers accept it?"** → Make it a coaching tool the worker sees first, opt-in, with no ranking shown to management.

Spoken summary (60 seconds)

"I'd agree with the customer what skill means in measurable terms: time, consistency, path efficiency, errors, ergonomics. Using the step segmentation from the cycle-discovery pipeline, I'd model the distribution of expert motion per step, then compare novices step by step and turn the biggest differences into specific coaching feedback. The score is only kept if it tracks real outcomes like supervisor ratings and defects, and I'd check fairness across handedness and body size. It's a coaching tool, not a ranking."

Your link: speakfrench benchmark (Wav2Vec2 embeddings beat DTW for scoring learners, ROC–AUC 0.822 vs. 0.599); McGill guitar case study with players.

O. Does motion predict downstream defects?

Prompt: "Quality finds defects at end-of-line. The customer wonders whether the operator's motion during assembly predicts which units will fail. How would you investigate?"

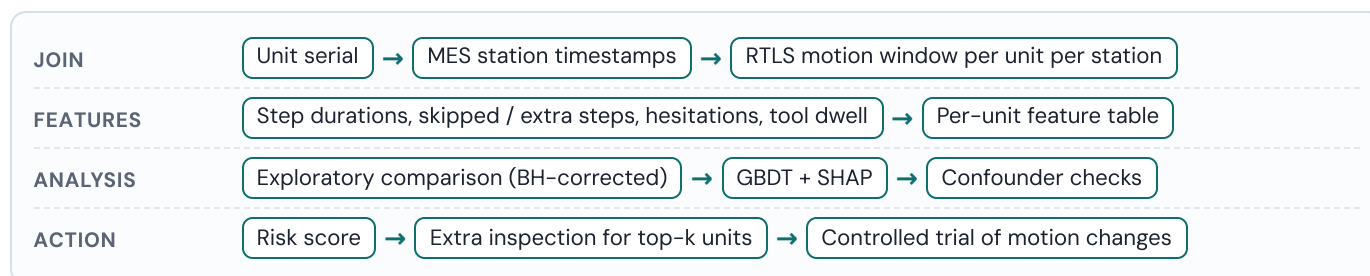
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Can units be linked to station timestamps?	Yes, serial numbers + MES timestamps.	Motion windows can be joined to outcomes per unit.
Defect rate?	~0.5%, about 30 defects per month.	Very few positives: simple models, careful statistics.
Defect types?	Mostly loose connectors and missing clips.	Link each type to the stations that could cause it.
What would they do with a prediction?	Send flagged units to extra inspection.	The metric is catch rate at a fixed inspection budget.

Requirements

- **Functional:** (1) an analysis answering "which motion patterns are associated with which defects"; (2) if the signal is real, a per-unit risk score used to route units to extra inspection.
- **Non-functional:** the score is available before the unit reaches end-of-line; explanations engineers can act on; statistically defensible claims.

Architecture



Components

- **Join the data:** each unit's time at each station → the operator's motion during exactly that window.
- **Features:** step durations (from the Scenario I pipeline), missing or repeated steps, hesitations, tool dwell, torque events if available.
- **Exploratory analysis first:** compare features between failed and passed units. With many features and stations, correct for **multiple comparisons** (Benjamini–Hochberg).
- **Model:** GBDT on per-unit features, with SHAP for explanations. Sequence models only if the signal is clearly there and data grows.
- **Confounders:** shift, operator experience, supplier lot, machine state. Include them as features or stratify; watch for **Simpson's paradox** (a pattern that correlates with defects only because it's common on night shift).

- **From correlation to action:** a controlled trial (coach a motion change on some stations, compare defect rates with difference-in-differences).

Budgets

- Data: ~3–6 months to have ~100–200 defects; do a power analysis before promising results.
- Engineer time: joint review of the top findings.

Evaluation

- **Time-based split:** train on earlier months, test on later ones.
- **Metrics:** PR-AUC; defects caught at a fixed inspection budget (e.g. top 2% of units) compared with random inspection (lift).
- **Robustness:** does the finding hold within each shift and each operator group?

Rollout

- **Month 1:** data join and exploratory analysis; report findings with honest uncertainty.
- **Month 2:** if a signal exists, a risk score in shadow mode.
- **Month 3+:** extra-inspection routing; controlled trial of process changes.

Follow-up questions, with answers

- **"We found it: long dwell at station 4 predicts defects."** → Could be reverse causation: operators pause *because* they noticed a bad part. Check with engineers and look at what happens during the dwell.
- **"Only 30 defects."** → Too few for complex models. Pool similar defect types, use simple interpretable models, be explicit about confidence intervals, or frame it as anomaly detection.
- **"Can we blame operators with this?"** → No: the analysis is about process design; many causes are upstream (parts, tools).

Spoken summary (60 seconds)

"First I'd join each unit's serial number to the operator's motion at each station. Then an exploratory comparison between failed and passed units, corrected for multiple comparisons, followed by a GBDT with SHAP explanations. The big risks are few defects and confounding: shift, experience, supplier lot, and even reverse causation. So I'd use time-based validation, stratify by confounders, and only claim a causal effect after a controlled trial. The practical output is a risk score that routes the riskiest units to extra inspection."

Your link: MAS changeover and allocation prediction; statistics in your user studies.

P. Tool-angle check from two tags

Prompt: "Drilling must be done within 5° of perpendicular to the panel. We can put two tags on the drill, 15 cm apart. Can we check the angle in real time?"

Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Accuracy needed?	Detect > 5°; ideally measure to ~1°.	Check whether the noise budget allows 1°.
Panel orientation known?	Yes, from the fixture CAD.	Compute the angle relative to the panel normal.
Drill trigger signal available?	Yes, from a smart tool controller.	Clean start and end of each drilling event.
Alert during or after drilling?	During, if possible.	Real-time at the edge.

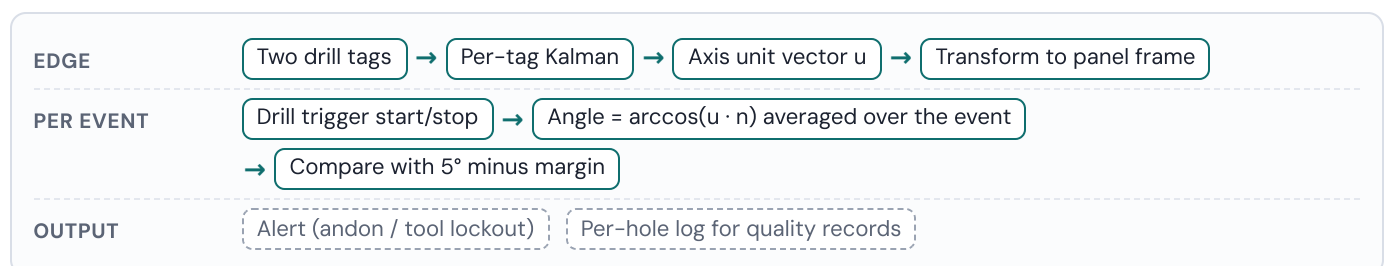
Requirements

- **Functional:** angle between the drill axis and the panel normal during each drilling event; alert if > 5°.
- **Non-functional:** accuracy $\leq 1^\circ$ (1σ) per event; alert within 200 ms; runs at the edge.

The key calculation (say it out loud)

- Each tag has $\sigma \approx 1.5$ mm. The difference vector between two tags has noise $\approx \sqrt{2} \times 1.5 \approx 2.1$ mm.
- Over a 150 mm baseline, the angle error $\approx 2.1 / 150$ rad ≈ 0.014 rad $\approx 0.8^\circ$ per sample (1σ).
- Averaging over a 1 s drilling event at 20 Hz (*assume*): $0.8 / \sqrt{20} \approx 0.2^\circ$, if the errors are independent (they partly aren't, so treat it as a best case).
- **Conclusion:** feasible. Two tags give the axis direction but not roll around it, and roll doesn't matter for perpendicularity.

Architecture



Components

- **Axis vector:** $u = (p_2 - p_1) / \|p_2 - p_1\|$, after the per-tag Kalman filter.
- **Tag-to-axis calibration:** the tags aren't exactly on the drill axis; calibrate once per tool with a jig (Kabsch/Procrustes fit between tag positions and known axis positions).
- **Panel normal:** from the fixture CAD, corrected by measuring three points on the real panel with a tagged probe.
- **Decision:** alert if the event-averaged angle > 5° - $k \cdot \sigma$ (a margin based on the noise estimate).
- **Where ML fits:** event detection if there's no trigger signal; later, learning which deviations actually cause defects.

Budgets

- Latency: geometry is microseconds; the budget is dominated by positioning (≤ 100 ms (*assume*)).
- Accuracy: $\sim 0.8^\circ$ per sample, $\sim 0.2\text{--}0.5^\circ$ per event.

Evaluation

- Jig tests at known angles (0° , 3° , 5° , 8°): report bias and spread.
- Live: compare with manual angle-gauge checks on a sample of holes.

Follow-up questions, with answers

- **"We need the roll angle too."** → Add a third, non-collinear tag; full orientation from Kabsch.
- **"The baseline can only be 5 cm."** → Error triples to $\sim 2.4^\circ$ per sample; averaging still gets under 1° per event, but it's marginal, so say that.
- **"Where's the ML?"** → Mostly geometry. Not every problem needs deep learning, and saying so is a strength. ML helps with event detection and learning which deviations matter.

Spoken summary (60 seconds)

"This is mostly a geometry problem. The two tags give the drill axis; the angle to the panel normal is arccos of the dot product. Noise: each tag is about 1.5 mm, the difference about 2.1 mm, over 150 mm that's roughly 0.8 degrees per sample, and averaging over a one-second event brings it well under a degree. So it's feasible. I'd calibrate the tag-to-axis offset once per tool with a jig, use the drill trigger for events, and validate with jig tests at known angles. ML only comes in for event detection and learning which deviations actually cause defects."

Your link: Promptly image registration and warping (applied geometry).

Q. PPE compliance from cameras

Prompt: "A plant wants to detect workers not wearing helmets, vests or gloves in certain zones, using existing CCTV. How would you build it?"

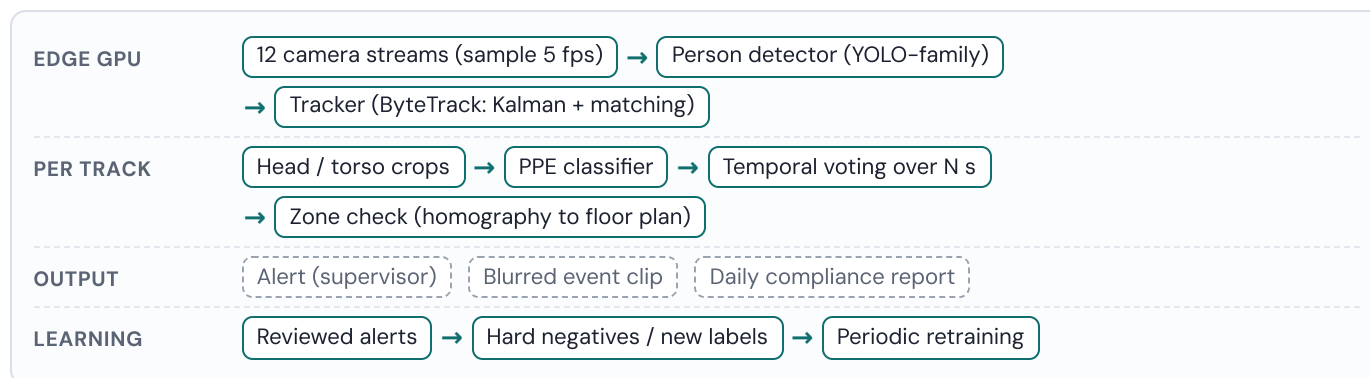
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Cameras?	12 fixed CCTV cameras, 1080p, 15 fps, high-mounted.	Small people in frame; gloves will be hard.
Which PPE matters most?	Helmets and vests; gloves in two zones.	Start with helmets and vests; gloves later with closer cameras.
Real-time or reports?	Real-time for restricted zones; daily reports elsewhere.	Edge inference for alerts.
Privacy?	Faces must not be stored.	Blur faces; store events and body crops only.

Requirements

- **Functional:** for each person in a PPE-required zone: helmet yes/no, vest yes/no; alert if missing for > N seconds; daily compliance report.
- **Non-functional:** alert within 5 s; < 1 false alert per camera per day (*target to agree*); runs on an edge GPU (e.g. an NVIDIA Jetson-class device); no face storage.

Architecture



Components

- **Person detection:** a YOLO-family detector, fine-tuned on site images.
- **Tracking:** ByteTrack or DeepSORT (Kalman prediction + Hungarian matching), so each **person** is judged over time, not each frame.
- **PPE classification per track:** head and torso crops into a small classifier, or a detector with PPE classes. Gloves are small and often occluded; treat them as a later phase with closer cameras.
- **Temporal voting:** "no helmet" only if consistent for N seconds of the track. This removes single-frame errors (helmet occluded, head turned).
- **Zones:** a homography from each camera to the floor plan maps each person's feet to a zone.
- **Privacy:** blur faces in any stored clip; keep events, not continuous video; short retention.

Budgets

- Compute: 12 cameras × 5 fps = 60 frames/s of detection plus classification of crops. Plausible on one edge GPU with TensorRT and a small model (*benchmark before promising*).
- Labels: public PPE datasets for pre-training, plus ~2,000–5,000 labelled site crops (*assume*), weighted toward hard cases.

Evaluation

- **Split by camera and by day**, never by frame (frames of one clip are near-duplicates).
- Event-level precision and recall per PPE type; false alerts per camera per day.
- **By condition:** day/night, distance from camera, crowding.

Rollout

- **Weeks 1–2:** record and label site data; benchmark detector and classifier.
- **Month 1:** shadow mode; review every alert.
- **Month 2:** live alerts in restricted zones; daily reports elsewhere.

Follow-up questions, with answers

- **"Night-shift lighting is poor."** → Augmentation, per-camera calibration, IR cameras where needed; report performance by lighting condition.
- **"Caps and hoods look like helmets."** → Add them as hard negatives; review false positives weekly.
- **"Could ZeroKey tags do this instead?"** → Only if the PPE is tagged (tagged helmets). Cameras catch untagged items; RTLS gives precise location. The hybrid is a strong answer.

Spoken summary (60 seconds)

"I'd judge people, not frames: detect people, track them with a Kalman-based tracker, classify helmet and vest on head and torso crops, and only alert when the violation is consistent for several seconds inside a zone mapped from the camera to the floor plan. It runs on an edge GPU at a reduced frame rate. The data is split by camera and day to avoid leakage, and I'd measure false alerts per camera per day, broken down by lighting. Gloves are a later phase with closer cameras, and faces are blurred in anything stored."

Your link: MAS industrial CV with alert budgets (5–6 s). Be honest that your hands-on CV was classical (see the gap script in the technical guide).

R. New product, 20 defect images

Prompt: "A new product line starts next month. Quality has thousands of good-part images but only about 20 defect images. How would you build visual inspection?"

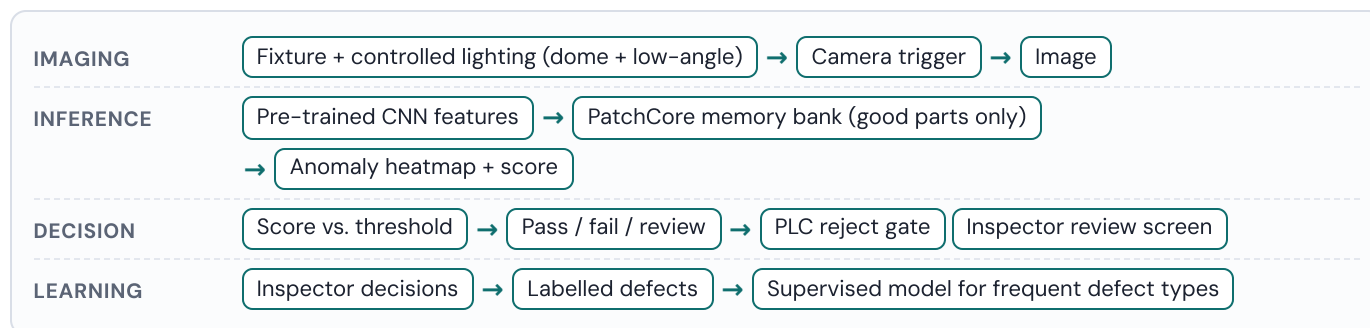
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
Defect types?	Scratches, dents, missing clips.	Scratches need low-angle lighting; missing clips are easy.
Cycle time per part?	6 s.	Inference must fit in well under 6 s.
Acceptable false-reject rate?	< 1% (rejects are re-inspected by hand).	Sets the threshold.
Imaging setup?	Not decided yet.	Big opportunity: fix lighting and pose first.

Requirements

- **Functional:** pass / fail / review per part, with a heatmap of the suspected defect.
- **Non-functional:** decision within 1 s; recall on critical defects near 100%; false-reject rate < 1%; explainable to QA.

Architecture



Components

- **Imaging first:** fixed pose via a fixture; diffuse dome lighting for general defects and low-angle lighting for scratches and dents. This often matters more than the model.
- **Anomaly detection on good parts only:** PatchCore (a memory bank of patch features from a pre-trained CNN) or PaDiM. Output is a heatmap, which makes it explainable to QA.
- **The 20 defect images are for validation and setting the threshold**, not training. Add **synthetic defects** (cut-and-paste, drawn scratches) for more validation.
- **Three-way decision:** pass, fail, or "review" by an inspector for scores near the threshold.
- **Later:** once real defects accumulate, a supervised classifier or segmenter for the frequent types, keeping the anomaly model for new defect types.

Budgets

- Latency: PatchCore on a GPU is typically well under a second per image (*benchmark*); fits the 6 s cycle.

- Labelling: none up front; inspectors' decisions become labels.

Evaluation

- Image-level AUROC and pixel-level metrics (MVTec-style) on the good images plus 20 real and many synthetic defects.
- **Operating point:** false-reject rate at 100% recall on critical defects.
- Bootstrap confidence intervals, since 20 defects is a tiny test set.

Rollout

- **Before launch:** design lighting and fixture; collect good-part images; validate on the 20 defects.
- **First weeks of production:** conservative threshold (favour "review"); every rejected and reviewed part checked by an inspector.
- **Month 2+:** tighten thresholds; add supervised models for frequent defects.

Follow-up questions, with answers

- **"Good parts vary a lot — three colours."** → Separate memory banks per variant, or condition on variant.
- **"A threshold from 20 defects is unreliable."** → Agreed: bootstrap intervals, start conservative with a review queue, tighten as real defects accumulate.
- **"Why not train a classifier on the 20?"** → Too few, and it would only learn those defect types. Anomaly detection generalizes to defects it has never seen.

Spoken summary (60 seconds)

"With thousands of good images and 20 defects, I'd flip the problem: model what 'good' looks like with an anomaly detector like PatchCore, and use the 20 defects only to validate and set the threshold. Before any model, I'd fix the imaging: fixture and lighting, low-angle for scratches. The output is pass, fail or review, with a heatmap for QA, and inspectors' decisions become labels for supervised models later. I'd start conservative, because a missed critical defect costs far more than a re-inspection."

Your link: MAS care-label QC and fabric defect detection (camera arrays, lighting lessons, PLC reject); Xu Han's quality-inspection work.

S. AGV fleet congestion forecasting

Prompt: "A plant runs 40 AGVs and hundreds of workers on shared aisles. AGVs often stop for congestion. Using RTLS for vehicles and people, can we predict congestion 30–60 s ahead?"

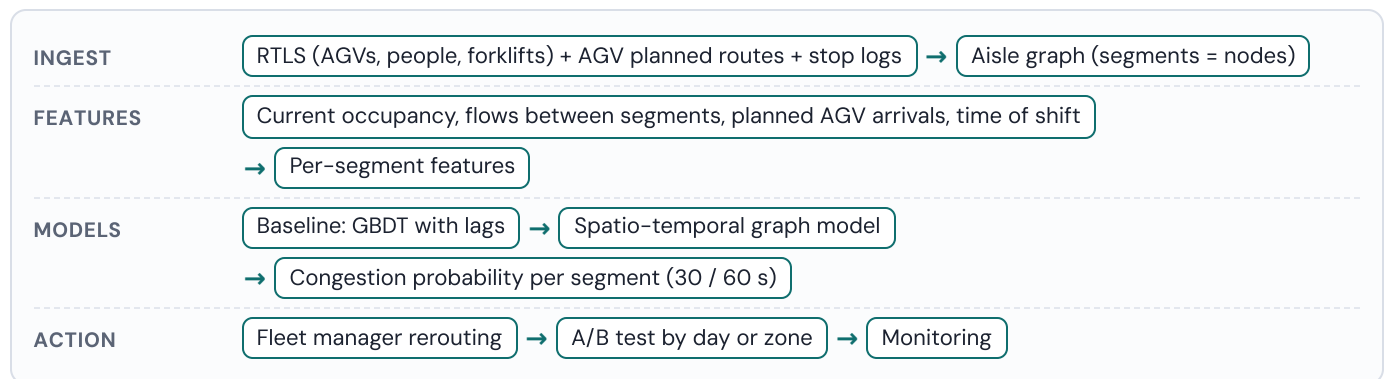
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the design
What happens with a prediction?	The fleet manager can reroute or delay dispatch.	Output must plug into the fleet manager's API.
Planned AGV routes available?	Yes, from the fleet manager.	Future AGV positions are mostly known; the uncertainty is people and forklifts.
How is "congestion" defined?	An AGV stopped > 10 s due to an obstacle.	Clear event label from AGV logs.
Data history?	3 months.	Enough for forecasting models with daily and shift seasonality.

Requirements

- **Functional:** probability of congestion per aisle segment at 30 s and 60 s ahead; fed to the fleet manager.
- **Non-functional:** update every 5 s; inference < 1 s; improvement measured in AGV stop time, not only forecast accuracy.

Architecture



Components

- **Aisle graph:** segments of aisles as nodes, connections as edges; each node holds occupancy by agent type.
- **Baseline:** GBDT on current occupancy, recent flows, planned AGV arrivals, and time of shift. Often strong.
- **Graph model:** a spatio-temporal graph network (the traffic-forecasting family, e.g. DCRNN or Graph WaveNet) captures how congestion spreads between neighbouring segments.
- **Human motion near intersections:** a constant-velocity forecast as a baseline, learned trajectory forecasting if needed.
- **Calibration:** probabilities must be calibrated so the fleet manager can set its own threshold.

Evaluation

- **Offline:** rolling-origin validation; precision and recall of congestion events at 30 and 60 s; calibration (reliability diagram).
- **Online:** an A/B test by day or zone, with total AGV stop time and throughput as the outcome.

Rollout

- **Month 1:** baseline GBDT, offline evaluation.
- **Month 2:** shadow predictions alongside the fleet manager.
- **Month 3:** A/B test rerouting on some zones.

Follow-up questions, with answers

- **"Rerouting prevents the congestion we'd have seen — how do you evaluate?"** → That's a feedback loop: historical accuracy stops being meaningful after deployment. Evaluate with experiments (A/B by zone or day) on the business metric.
- **"Isn't this RL?"** → The routing policy could be learned with RL eventually, offline from logged data. Start with prediction plus the existing planner; it's cheaper and safer.

Spoken summary (60 seconds)

"I'd represent the aisles as a graph, with occupancy and flows per segment for AGVs, people and forklifts, plus the AGVs' planned routes, which make most of the future known. A GBDT baseline on those features, then a spatio-temporal graph model to capture how congestion spreads, predicting a calibrated probability per segment 30 and 60 seconds ahead. The fleet manager uses it to reroute, and because rerouting changes the data, I'd judge success with an A/B test on AGV stop time, not historical accuracy."

Your link: multi-entity stream sync (MUSMET); MAS allocation prediction.

T. "Our model is 98% accurate" — find the problem

Prompt: "A previous team built a risky-behaviour classifier on RTLS data. They report 98% accuracy, but in the pilot the operators say it's useless. Here's their setup: 2-second windows with 50% overlap, random 80/20 split, accuracy metric, 3% of windows labelled risky. What's wrong, and what would you do?"

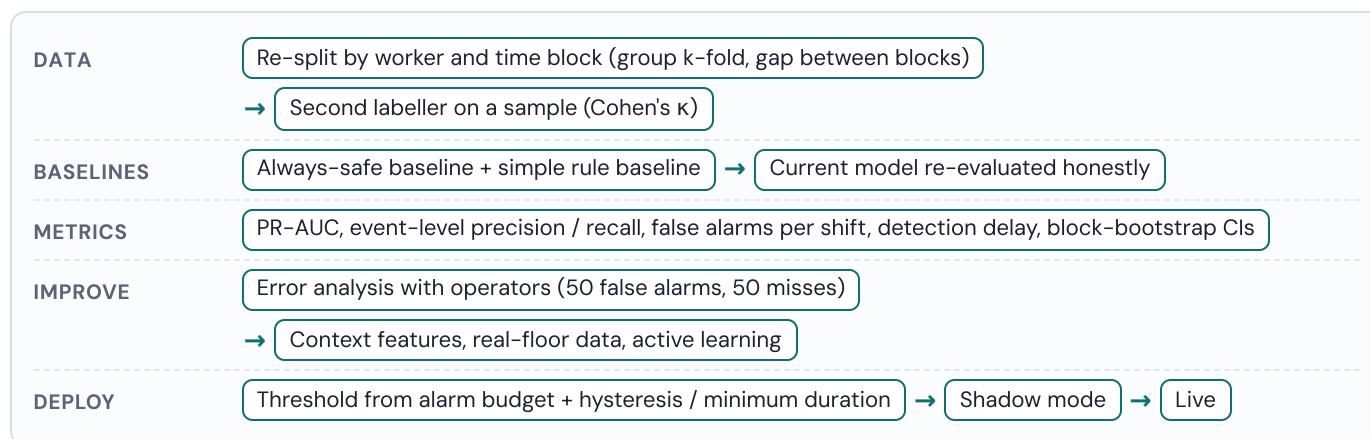
Clarifying questions, with likely answers

You ask	Likely answer	How it changes the answer
How were labels made?	One engineer labelled video of staged sessions.	Label noise, and staged $\neq$ real.
How many workers and sessions?	8 workers, 12 sessions.	Few groups; person-level leakage is likely.
What do operators complain about?	Too many false alarms, and it misses obvious events.	Both precision and recall are poor in reality.
What threshold?	Default 0.5.	Not chosen for an alarm budget.

What's wrong (the core of the answer, list it clearly)

1. **Accuracy under imbalance:** with 3% positives, always predicting "safe" scores 97%. 98% is barely above trivial.
2. **Leakage from overlapping windows:** with 50% overlap and a random split, neighbouring windows sharing half their samples land in both train and test.
3. **Leakage by person and session:** the same workers and sessions are in train and test, so the model can memorize individuals.
4. **Window-level instead of event-level:** operators experience alerts per event; one risky event spans many windows.
5. **Threshold:** default 0.5, not chosen for an alarm budget.
6. **Lab vs. floor:** staged data differs from real work (distribution shift).
7. **Labels:** a single labeller, no agreement measure.

The fix, as a system



Evaluation after the fix

- Report every metric against the always-safe baseline and the rule baseline.
- Group k-fold by worker; confidence intervals with a block bootstrap.
- Shadow mode on real shifts before any alerts go live.

Follow-up questions, with answers

- **"After fixing the split, F1 drops to 0.4. Now what?"** → That's the real starting point, and it's good to know. Check label quality, add context features (zone, task, SOP step), use rules for the clearest risks, collect real-floor data, and use active learning.
- **"Management liked 98%. How do you explain this?"** → Show the always-safe baseline next to it: "a model that never alerts also scores 97%." Then show event-level metrics the operators recognize.
- **"Which one issue matters most?"** → Leakage, because it makes every other number meaningless.

Spoken summary (60 seconds)

"98% accuracy with 3% positives is almost the same as never alerting, which scores 97%. On top of that, overlapping windows with a random split leak near-duplicate data into the test set, and the same workers appear in train and test, so the number measures memorization. I'd re-split by worker and time with gaps, compare against an always-safe and a rule baseline, report event-level precision and recall and false alarms per shift, choose the threshold from the alarm budget, and do error analysis with the operators. The honest number will be lower, and that's the real starting point."

Your link: JAES honesty (F1 0.76, reported against a DTW baseline); "a suspiciously high score usually means leakage."

Part 3 — Reusable lines

- "Before modelling, I'd check what a simple rule gets us — that's the bar any ML has to beat."
- "At 20 Hz — assuming that rate — a 2-second window is 40 samples."
- "I'd evaluate at the event level, split by worker and site, because that's what deployment looks like."
- "The cheapest experiment first: record a static reference tag and label one shift with an engineer."
- "If operators don't trust it, accuracy doesn't matter — explainability and a low false-alarm rate are part of the spec."
- "Edge for anything safety-critical or latency-bound; server for learning, analytics and the agent."
- "Every model becomes a tool the OmniVisor agent can call, returning numbers you can check."
- "Good point — if that's true, I'd change X to Y, because..."